

6

أنواع البيانات والتحويل بينها

تُصنّف **بايثون** البيانات (data) إلى أنواع، كما هو الحال في جميع لغات البرمجة. هذا مهم لأنّ نوع البيانات الذي تستخدمه سيقيّد القيم التي يمكن تعيينها لها، وما الذي يمكن فعله بها (بما في ذلك العمليات التي يمكن تنفيذها عليها).

سنتعرّف في هذا الفصل على أهم أنواع البيانات الأصليّة ل**بايثون**. هذا ليس استقصاءً شاملاً لأنواع البيانات، ولكنّه سيساعدك على التعرف على الخيارات المتاحة لك في **بايثون**.

1. خلفية عامة

أنواع البيانات في **بايثون** مشابهة إلى حد ما لأنواع البيانات التي نستخدمها في العالم الحقيقي. من أمثلة أنواع البيانات في العالم الحقيقي الأعداد، مثل: الأعداد الصحيحة الطبيعية (0, 1, 2, ...)، والأعداد الصحيحة النسبية (...، -1، 0، 1، ...)، والأعداد غير النسبية (π). يمكننا عادة في الرياضيات جمع أعداد من أنواع مختلفة مثل إضافة 5 إلى π :

$$5 + \pi$$

يمكننا إمّا الاحتفاظ بالمعادلة بعدّها إجابة، وستكون النتيجة عدداً غير نسبي (irrational number)، أو يمكننا تقريب (round) العدد π إلى عدد ذي منازل عشرية محددة، ثم نجمع العددين:

$$5 + \pi = 5 + 3.14 = 8.14$$

ولكن، إذا حاولنا إضافة عدد إلى نوع بيانات آخر، مثل الكلمات، فستصبح الأمور مربكة وغير ذات معنى. فكيف ستحل المعادلة التالية مثلاً؟

$$\text{hsoub} + 8$$

بالنسبة إلى الكلمة `hsoub`، يمكن عدُّ كل نوع من أنواع البيانات مختلفًا تمامًا، مثل الكلمات والأعداد، لذلك يتعيَّن علينا توخي الحذر بشأن كيفية استخدامها، وكيفية التعامل معها في العمليات.

2. الأعداد

سيُفسَّر كل عدد تُدخله إلى **بايثون** على أنه عدد؛ ليس مطلوبًا منك الإعلان صراحةً عن نوع البيانات الذي تدخله لأنَّ **بايثون** تعدُّ أيَّ عدد مكتوب بدون فواصل عشرية بمثابة عدد صحيح (`integer`، كما هو حال 138)، وأيُّ عدد مكتوب بفواصل لعشرية بمثابة عدد عشري (`float`، كما هو حال 138.0).

١. الأعداد الصحيحة (`integer`)

كما هو الحال في الرياضيات، **الأعداد الصحيحة** (`integer`) في البرمجة هي أعداد كاملة، يمكن أن تكون موجبة أو سالبة أو معدومة (...، 1-، 0، 1، ...). ويُعرف هذا النوع أيضًا باسم `int`. كما هو الحال مع لغات البرمجة الأخرى، يجب ألا تستخدم الفواصل في الأعداد المؤلفة من أربعة أرقام أو أكثر، لذلك لا تكتب 1,000 في برنامجك، واكتب 1000.

يمكننا طباعة عدد صحيح على النحو التالي:

```
print(-25)
```

وسينتج:

```
-25
```

أو يمكننا الإعلان عن متغير، والذي هو في هذه الحالة رمزٌ للعدد الذي نستخدمه أو نتعامل

معه، مثلًا:

```
my_int = -25
print(my_int)
```

وسينتج لنا:

```
-25
```

يمكننا أن نجري العمليات الحسابية على **الأعداد الصحيحة** في بايثون مباشرةً مثل:

```
int_ans = 116 - 68
print(int_ans)
```

المخرجات:

```
48
```

يمكن استخدام الأعداد الصحيحة بعدة طرائق في برامج بايثون، ومع استمرارك في تعلم المزيد عن هذه اللغة، ستتاح لك الكثير من الفرص لاستخدام الأعداد الصحيحة والتعامل معها وفهم المزيد عن هذا النوع من البيانات.

ب. الأعداد العشرية (Floating-Point Numbers)

الأعداد العشرية هي أعداد حقيقية، مما يعني أنه يمكن أن تكون أعدادًا جذرية أو غير

نسبية. لهذا السبب، يمكن أن تحتوي الأعداد العشرية على جزء كسري، مثل 9.0 أو -116.42.

وببساطة، فالأعداد العشرية هي أعداد تحتوي الفاصلة العشرية.

كما فعلنا مع الأعداد الصحيحة، يمكننا طباعة الأعداد العشرية هكذا:

```
print(17.3)
```

وسينتج لنا:

```
17.3
```

يمكننا أيضًا أن نعلن عن متغيرٍ يحوى عددًا عشريًا، مثلًا:

```
myflt = 17.3
print(myflt)
```

الناتج:

```
17.3
```

وكما هو الحال مع الأعداد الصحيحة، يمكننا أن نجرى العمليات الحسابية على

الأعداد العشرية:

```
flt_ans = 564.0 + 365.24
print(flt_ans)
```

الناتج:

```
929.24
```

تختلف الأعداد الصحيحة والأعداد العشرية عن بعضها عمومًا، إذ أن $3.0 \neq 3$ ، لأن 3 عدد

صحيح، بينما 3.0 عدد عشري.

3. القيم المنطقية

هناك قيمتان فقط لنوع **البيانات المنطقية** (Boolean)، وهما True و False. تُستخدم القيم

المنطقية لتمثيل قيم الحقيقة الموافقة للمنطق الرياضي (صح أو خطأ).

عادة ما يبدأ اسم البيانات المنطقية بالحرف B، إشارة إلى اسم عالم الرياضيات George Boole. القيمتان True و False تُكتبان دائمًا بحرفين كبيرين T و F، لأنها قيم خاصة في بايثون. الكثير من العمليات الحسابية في الرياضيات تُنتج قيمًا منطقيًا، إما True أو False:

- أكبر من

```
bool_val = 500 > 100 # True
bool_val = 1 > 5     # False
```

- أصغر من

```
bool_val = 200 < 400 # True
bool_val = 4 < 2     # False
```

- التساوي

```
bool_val = 5 = 5      # True
bool_val = 500 = 400 # False
```

كما هو الحال مع الأعداد، يمكننا تخزين القيم المنطقية في المتغيرات:

```
my_bool = 5 > 8
```

يمكننا بعد ذلك طباعة القيمة المنطقية باستدعاء الدالة `print()`:

```
print(my_bool)
```

بما أنَّ العدد 5 ليس أكبر من 8، فسوف نحصل على المخرجات التالية:

```
False
```

ستتعلم مع مرور الوقت كيفية استخدام القيم المنطقية، وكيف يمكن للدوال والعمليات المنطقية أن تغيّر مسار البرنامج.

4. السلاسل النصية

السلسلة النصية (string) هي عبارة عن تسلسل من محرف واحد أو أكثر (محارف وأعداد ورموز)، ويمكن أن تكون ثابتة أو متغيرة. تحاط السلاسل النصية إما بعلامات الاقتباس المفردة ' أو علامات الاقتباس المزدوجة "، لذلك لإنشاء سلسلة نصية، ضع سلسلة من الأحرف بين علامتي اقتباس:

```
'This is a string in single quotes.'  
"This is a string in double quotes."
```

يمكنك استخدام علامات الاقتباس المفردة أو علامات الاقتباس المزدوجة، المهم أن تكون متسقًا في برنامجك.

البرنامج البسيط "Hello, World!" يوضح كيف يمكن استخدام السلاسل النصية في البرمجة، إذ أنّ حروف عبارة Hello, World! تمثل سلسلة نصية.

```
print("Hello, World!")
```

كما هو الحال مع أنواع البيانات الأخرى، يمكننا تخزين السلاسل النصية في المتغيرات:

```
hw = "Hello, World!"
```

وطباعة السلسلة عن طريق استدعاء المتغيّر:

```
print(hw) # Hello, World!
```

مثل الأعداد، هناك العديد من العمليات التي يمكن إجراؤها على السلاسل النصية من أجل تحقيق النتائج التي نسعى إليها. السلاسل النصية مهمة لتوصيل المعلومات إلى المستخدم، وكذلك لتمكين المستخدم من تمرير المعلومات إلى البرنامج.

5. القوائم (Lists)

القائمة (list) عبارة عن تسلسل مرتّب قابل للتغيير (mutable). وكما تُعرّف السلاسل النصيّة باستخدام علامات الاقتباس، يتم تعريف القوائم باستخدام الأقواس المعقوفة []. مثلاً، هذه قائمة تحوي أعدادًا صحيحة:

```
[-3, -2, -1, 0, 1, 2, 3]
```

وهذه قائمة من الأعداد العشرية:

```
[3.14, 9.23, 111.11, 312.12, 1.05]
```

وهذه قائمة من السلاسل النصية:

```
['shark', 'cuttlefish', 'squid', 'mantis shrimp']
```

في المثال التالي، سنسمّي قائمة السلاسل النصية خاصتنا بالاسم `sea_creatures`:

```
sea_creatures = ['shark', 'cuttlefish', 'squid', 'mantis shrimp']
```

يمكننا طباعتها عن طريق استدعاء المتغير:

```
print(sea_creatures)
```

وسترى أنّ المخرجات تشبه تمامًا القائمة التي أنشأناها:

```
['shark', 'cuttlefish', 'squid', 'mantis shrimp']
```


القوائم هي نوع بيانات مرنة للغاية، لأنها قابلة للتغيير، حيث يمكن إضافة قيم إليها، أو إزالتها، أو تغييرها. هناك نوع بيانات آخر مشابه لقوائم، يُدعى أنه غير قابل للتغيير، ويُسمى الصف (tuple).

6. الصفوف (Tuples)

يُستخدم الصف (tuple) لتجميع البيانات، وهو تسلسل ثابت من العناصر وغير قابل للتغيير. الصفوف تشبه القوائم إلى حد كبير، لكنها تستخدم الأقواس () بدلاً من الأقواس المعقوفة []، ولأنها غير قابلة للتغيير، فلا يمكن تغيير أو تعديل قيمها. تبدو الصفوف كالتالي:

```
('blue coral', 'staghorn coral', 'pillar coral')
```

يمكننا تخزين الصفوف في المتغيرات وطباعتها:

```
coral = ('blue coral', 'staghorn coral', 'pillar coral')
print(coral)
```

والمخرجات هي:

```
('blue coral', 'staghorn coral', 'pillar coral')
```

كما هو الحال في أنواع البيانات الأخرى، تطبع بايثون الصفوف تمامًا كما كتبناها، إذ تطبع سلسلة من القيم بين قوسين.

7. القواميس (Dictionaries)

القاموس (dictionary) هو نوع مُضمّن في بايثون، إذ تُربط مفاتيحُ بالقيم المقابلة لها في شكل أزواج، هذه الأزواج مفيدة لتخزين البيانات في بايثون. يتم إنشاء القواميس باستخدام الأقواس المعقوفة {}.

تُستخدم القواميس عادةً لحفظ البيانات المترابطة، مثل المعلومات المقابلة لرقم تعريف. يبدو القاموس كما يلي:

```
{'name': 'Sammy', 'animal': 'shark', 'color': 'blue',  
'location': 'ocean'}
```

ستلاحظ أنه بالإضافة إلى الأقواس المعقوفة، توجد علامات النقطتين الرأسيتين (colons) داخل القاموس. الكلمات الموجودة على يسار النقطتين الرأسيتين هي المفاتيح. المفاتيح قد تكون أيّ نوع بيانات غير قابل للتغيير. المفاتيح في القاموس أعلاه هي:

```
'name', 'animal', 'color', 'location'
```

الكلمات الموجودة على يمين النقطتين هي القيم. يمكن أن تتألف القيم من أي نوع من البيانات. القيم في القاموس أعلاه هي:

```
'Sammy', 'shark', 'blue', 'ocean'
```

مثل أنواع البيانات الأخرى، يمكننا تخزين القواميس في متغيرات، وطباعتها:

```
sammy = {'name': 'Sammy', 'animal': 'shark', 'color': 'blue',  
'location': 'ocean'}  
print(sammy)
```

والمخرجات هي:

```
{'color': 'blue', 'animal': 'shark', 'name': 'Sammy',  
'location': 'ocean'}
```

إذا أردت الحصول على اللون (color) الخاص بـ Sammy، فيمكنك القيام بذلك عن طريق

استدعاء ['color'] sammy. هذا مثال على ذلك:

```
print(sammy['color']) # blue
```

القواميس من أنواع البيانات المهمة في برامج بايثون.

8. التحويل بين أنواع البيانات

يحدّد نوع البيانات -كما ذكرنا- القيم التي يمكن استعمالها، والعمليات التي يمكنك إجراؤها

عليها. هناك أوقات نحتاج إلى تحويل القيم من نوعٍ إلى آخر لأجل معالجتها بطريقة مختلفة.

على سبيل المثال، قد نحتاج إلى ضم (concatenate) القيم العددية إلى سلاسل نصية.

سيرشدك هذا القسم إلى كيفية التحويل بين الأعداد والسلاسل النصية والصفوف والقوائم،

بالإضافة إلى تقديم بعض الأمثلة التوضيحية.

1. تحويل الأنواع العددية

هناك نوعان من البيانات العددية في بايثون كما رأينا آنفًا: **الأعداد الصحيحة والأعداد**

العشرية. ستعمل في بعض الأحيان على شيفرة برمجية كتبها شخص آخر، وقد تحتاج إلى

تحويل عدد صحيح إلى عدد عشري، أو العكس، أو قد تجد أنك تستخدم عددًا صحيحًا في

الوقت الذي تحتاج إلى أعداد عشرية. يتوفر في بايثون توابع مضمّنة تُسهّل عليك تحويل

الأعداد الصحيحة إلى أعداد عشرية، أو العكس.

تحويل الأعداد الصحيحة إلى أعداد عشرية

يحوّل التابع `float()` الأعداد الصحيحة إلى أعداد عشرية. لاستخدام هذه الدالة، ضع عددًا صحيحًا بين القوسين:

```
float(57)
```

في هذه الحالة، سيحوّل العدد الصحيح 57 إلى العدد العشري 57.0. يمكنك أيضًا استخدام هذه الدالة مع المتغيرات. لُسنِد القيمة 57 إلى المتغيّر `f`، ثم نطبع العدد العشري الجديد:

```
f = 57

print(float(f))
```

النتاج سيكون:

```
57.0
```

يمكننا باستخدام الدالة `float()` تحويل الأعداد الصحيحة إلى أعداد عشرية.

تحويل الأعداد العشرية إلى أعداد صحيحة

تملك بايثون دالة أخرى مضمّنة لتحويل الأعداد عشرية إلى أعداد صحيحة: وهي `int()`. تعمل الدالة `int()` بشكل مشابه للدالة `float()`: يمكنك إضافة عدد عشري داخل القوسين لتحويله إلى عدد صحيح:

```
int(390.8)
```

في هذه الحالة، سيحوّل العدد العشري 390.8 إلى العدد الصحيح 390.

يمكنك أيضًا استخدام هذه الدالة مع المتغيرات. لنصّرّح أنّ b يساوي 125.0 ، وأنّ c يساوي 390.8 ، ثم نطبع العددين العشريين الجديدين:

```
b = 125.0

c = 390.8

print(int(b))

print(int(c))
```

والمخرجات ستكون:

```
125
390
```

عند تحويل الأعداد العشرية إلى أعداد صحيحة بواسطة الدالة `int()`، تقتطع بايثون الأجزاء العشرية من العدد وتُبقي القيمة الصحيحة؛ لذلك، لن تُحوّل الدالة `int()` العدد 390.8 إلى 391 .

تحويل الأعداد عبر القسمة

في بايثون 3، عند تقسيم عدد صحيح على آخر، سينتج عدد عشري على خلاف بايثون 2. بمعنى أنّه عند قسمة 5 على 2 في بايثون 3، ستحصل على عدد عشري (مثل 2.5 عند قسمة 5 على 2):

```
a = 5 / 2

print(a)
```

وسينتج لنا:

```
2.5
```

بينما في بايثون 2، ستحصل على ناتج صحيح، أي $5/2 = 2$. يمكنك الحصول على عدد صحيح ناتج عن عملية القسمة باستعمال المعامل // الجديد في بايثون 3:

```
a = 5 // 2

print(a)
```

وسينتج لنا:

```
2
```

ارجع إلى فصل «إصدارات بايثون: بايثون 2 مقابل بايثون 3» للاطلاع على المزيد من الفروقات بين بايثون 2 وبايثون 3.

ب. التحويل مع السلاسل النصية

السلاسل النصية عبارة عن سلسلة مؤلفة من محرف واحد أو أكثر (المحرف يمكن أن يكون حرفاً، أو عدداً، أو رمزاً). السلاسل النصية هي إحدى الأشكال الشائعة من البيانات في عالم البرمجة، وقد نحتاج إلى تحويل السلاسل النصية إلى أعداد أو أعداد إلى سلاسل نصية في كثير من الأحيان، خاصةً عندما نعمل على البيانات التي ينشئها المستخدمون.

تحويل الأعداد إلى سلاسل نصية

يمكننا تحويل الأعداد إلى سلاسل نصية عبر التابع `str()`. يمكننا أن نمرّر إما عددًا أو متغيرًا بين قوسي التابع، وبعد ذلك سَتُحوَّل تلك القيمة العددية إلى قيمة نصية. دعنا ننظر أولاً في تحويل الأعداد الصحيحة. لتحويل العدد الصحيح 12 إلى سلسلة نصية، يمكنك تمرير 12 إلى التابع `str()`:

```
str(12)
```

عند تنفيذ `str(12)` في سطر أوامر بايثون التفاعلي مع الأمر `python` في نافذة الطرفية، ستحصل على المخرجات التالية:

```
'12'
```

تشير علامات الاقتباس المحيطة بالعدد 12 إلى أنه لم يعد عددًا صحيحًا، ولكنه أصبح الآن سلسلة نصية.

سيصبح باستخدام المتغيرات تحويل الأعداد الصحيحة إلى سلاسل نصية أكثر فائدة. لنفترض أننا نريد متابعة تقدُّم مستخدم في مجال البرمجة يوميًا مثل أن ندخل عدد أسطر الشيفرة البرمجية التي كتبها. نود أن نعرض ذلك على المستخدم، وذلك بطباعة السلاسل النصية والأعداد في الوقت نفسه:

```
user = "Sammy"

lines = 50

print("Congratulations, " + user + "! You just wrote " + lines
+ " lines of code.")
```

عند تنفيذ الشيفرة أعلاه، سيُطلق الخطأ التالي:

```
TypeError: Can't convert 'int' object to str implicitly
```

يتعذر علينا ضمُّ (concatenate) الأعداد إلى السلاسل النصية في بايثون، لذلك يجب

تحويل المتغير `lines` إلى سلسلة نصية:

```
user = "Sammy"
lines = 50

print("Congratulations, " + user + "! You just wrote " +
      str(lines) + " lines of code.")
```

الآن، عندما نُنفِّذ الشيفرة البرمجية، سنحصل على المخرجات التالية، وفيها تهنئة

للمستخدم على تقدُّمه:

```
Congratulations, Sammy! You just wrote 50 lines of code.
```

إذا أردنا تحويل عدد عشري إلى سلسلة نصية بدلاً من تحويل عدد صحيح إلى سلسلة

نصية، فعلينا تتبع نفس الخطوات والصياغة السابقة. عندما نمُرُّ عددًا عشريًا إلى التابع `str()`،

سُعاد سلسلة نصية. يمكننا استخدام قيمة العدد العشري نفسها، أو يمكننا استخدام متغيَّر:

```
print(str(421.034))
```

```
f = 5524.53
print(str(f))
```

وسينتج لنا:

```
421.034
5524.53
```


يمكننا اختبار صحة التحويل عن طريق ضم الناتج إلى سلسلة نصية:

```
f = 5524.53

print("Sammy has " + str(f) + " points.")
```

وهذا هو الناتج:

```
Sammy has 5524.53 points.
```

الآن تأكدنا من أنَّ عددنا العشري قد حُوِّل بنجاح إلى سلسلة نصية، لأنَّ عملية الضم قد تُفقد دون خطأ.

تحويل السلاسل النصية إلى أعداد

يمكن تحويل السلاسل النصية إلى أعداد باستخدام التابعين `float()` و `int()`. إذا لم يكن في السلسلة النصية منازل عشرية، فالأفضل أن تحولها إلى عدد صحيح باستخدام التابع `int()`.

دعنا نستخدم مثال تتبع عدد أسطر الشيفرة الذي أوردناه أعلاه. قد ترغب في التعامل مع هذه القيم باستخدام الحسابات الرياضية لتقديم نتائج أدق للمستخدم، ولكنَّ هذه القيم مخزَّنة حاليًّا في سلاسل نصية:

```
lines_yesterday = "50"

lines_today = "108"

lines_more = lines_today - lines_yesterday

print(lines_more)
```

الناتج هو:

```
TypeError: unsupported operand type(s) for -: 'str' and 'str'
```

نظرًا لأنَّ القيمتين العدديتين مخزَّنتان في سلاسل نصية، تلقينا خطأً. سبب ذلك أنَّ معامِل

الطرح - لا يصلح للسلاسل النصية.

دعنا نعدِّل الشيفرة لتضمين التابع `int()` الذي سيحول السلاسل النصية إلى أعداد

صحيحة، ويسمح لنا بالقيام بالعمليات الرياضية على القيم التي كانت سلاسل نصية في الأصل.

```
lines_yesterday = "50"

lines_today = "108"

lines_more = int(lines_today) - int(lines_yesterday)

print(lines_more)
```

وهذه هي المخرجات:

```
58
```

المتغير `line_more` هو عدد صحيح تلقائيًا، ويساوي القيمة العددية 58 في هذا المثال.

يمكننا أيضًا تحويل الأعداد في المثال أعلاه إلى قيم عشرية باستخدام التابع `float()` بدلًا

من التابع `int()`. وبدلًا من الحصول على الناتج 58، سنحصل على الناتج 58.0، وهو عدد عشري.

سيكسب المستخدم Sammy نقاطًا على شكل قيم عشرية:

```
total_points = "5524.53"

new_points = "45.30"

new_total_points = total_points + new_points

print(new_total_points)
```

النتيجة:

```
5524.5345.30
```

في هذه الحالة، يعدُّ استخدام المعامل + مع سلسلتين نصيتين عمليةً صالحةً، لكنه سيضم السلسلتين النصيتين بدلًا من جمع القيمتين العدديتين؛ لذلك، سيبدو الناتج غير مألوف، لأنَّه نتيجة لصق القيمتين إلى جانب بعضهما بعضًا. سنحتاج إلى تحويل هذه السلاسل النصية إلى أعداد عشرية قبل إجراء أي عمليات عليها، وذلك باستخدام التابع `float()`:

```
total_points = "5524.53"

new_points = "45.30"

new_total_points = float(total_points) + float(new_points)

print(new_total_points)
```

وسينتج عن ذلك:

```
5569.83
```

الآن، وبعد أن حوّلنا السلسلتين النصيتين إلى عددين عشريين، سنحصل على النتيجة المتوقعة، والتي هي جمع 45.30 و 5524.53.

إذا حاولنا تحويل سلسلة نصية ذات منازل عشرية إلى عدد صحيح، فسنحصل على خطأ:

```
f = "54.23"
print(int(f))
```

المخرجات:

```
ValueError: invalid literal for int() with base 10: '54.23'
```

إذا مرّرنا عددًا عشريًا موضوعًا في سلسلة نصية إلى التابع `int()`، فسنحصل على خطأ، إذ لن نُحوّل إلى عدد صحيح.

يتيح لنا تحويل السلاسل النصية إلى أعداد تعديل نوع البيانات الذي نعمل عليه بسرعة حتى نتمكن من إجراء عمليات على قيم عددية مكتوبة على شكل سلاسل نصية.

ج. التحويل إلى صفوف وقوائم

يمكنك استخدام التابعين `list()` و `tuple()` لتحويل القيم المُمرّرة إليهما إلى **قائمة** أو

صف على التوالي. في بايثون:

- القائمة هي تسلسل مرّتب قابل للتغيير من العناصر الموضوعة داخل قوسين معقوفين `[]`.
- الصف عبارة عن تسلسل مرتّب ثابت (غير قابل للتغيير) من العناصر الموضوعة بين القوسين الهلاليين `()`.

التحويل إلى صفوف

نظرًا لكون الصفوف غير قابلة للتغيير، فيمكن أن يحسّن تحويل قائمة إلى صف أداء البرامج تحسينًا كبيرًا. عندما نستخدم التابع `tuple()`، فسوف يُعيد القيمة المُمرّرة إليه على هيئة صف.

```
print(tuple(['pull request', 'open source', 'repository',
            'branch']))
```

المخرجات:

```
('pull request', 'open source', 'repository', 'branch')
```

نرى أنّ الصف قد طُبِع في المخرجات، إذ أنّ العناصر موضوعة الآن بين قوسين، بدلًا من القوسين المربعين.

دعنا نستخدم `tuple()` مع متغير يحتوي قائمة:

```
sea_creatures = ['shark', 'cuttlefish', 'squid', 'mantis
                 shrimp']

print(tuple(sea_creatures))
```

سيُنتج:

```
('shark', 'cuttlefish', 'squid', 'mantis shrimp')
```

مرة أخرى، نرى أنّ القائمة حُوِّلَت إلى صف، كما يشير إلى ذلك القوسان. يمكننا تحويل أي نوع قابل للتكرار (iterable) إلى صف، بما في ذلك السلاسل النصية:

```
print(tuple('Sammy'))
```

المخرجات:

```
('S', 'a', 'm', 'm', 'y')
```

لَمَّا كان بالإمكان المرور (iterate) على محارف السلاسل النصية، فيمكننا تحويلها إلى صفوف باستخدام التابع `tuple()`. أمَّا أنواع البيانات غير القابلة للتكرار، مثل الأعداد الصحيحة والأعداد العشرية، فسُطِّلِقَ عملية تحويلها خطأً:

```
print(tuple(5000))
```

والناتج سيكون:

```
TypeError: 'int' object is not iterable
```

في حين أنه من الممكن تحويل عدد صحيح إلى سلسلة نصية، ومن ثم تحويل السلسلة النصية إلى صف، كما في `tuple(str(5000))`، فمن الأفضل تجنب مثل هذه التعليمات البرمجية المعقَّدة.

التحويل إلى قوائم

يمكن أن يكون تحويل القيم، وخاصة الصفوف، إلى قوائم مفيدًا عندما تحتاج إلى نسخة قابلة للتغيير من تلك القيم.

سنستخدم التابع `list()` لتحويل الصف التالي إلى قائمة. ونظرًا لأنَّ صياغة القوائم تستخدم الأقواس، تأكد من تضمين أقواس التابع `list()`، وكذلك الأقواس الخاصة بالدالة `print()`:

```
print(list(('blue coral', 'staghorn coral', 'pillar coral')))
```

المخرجات هي:

```
['blue coral', 'staghorn coral', 'pillar coral']
```

تشير الأقواس المعقوفة [] إلى أنه قد أُرِجِعَت قائمة من الصف الأصلي الذي مُرِّرَ عبر

الدالة `list()`.

لجعل الشيفرة سهلة القراءة، يمكننا إزالة أحد أزواج الأقواس باستخدام متغيّر:

```
coral = ('blue coral', 'staghorn coral', 'pillar coral')

list(coral)
```

إن طبعنا `list(coral)`، فسنلتقى المخرجات نفسها الموجودة أعلاه.

تمامًا مثل الصفوف، يمكن تحويل السلاسل النصية إلى قوائم:

```
print(list('shark'))
```

النتاج:

```
['s', 'h', 'a', 'r', 'k']
```

حُوِّلَت هنا السلسلة `shark` إلى قائمة، وهذا يوفر لنا نسخة قابلة للتغيير من القيمة الأصلية.

9. خلاصة الفصل

في هذه المرحلة، يُفترض أن يكون لديك فهم جيد لبعض أنواع البيانات الرئيسية المتاحة

في بايثون. أنواع البيانات هذه ستصبح جزءًا طبيعيًا من حياتك كمبرمج للغة بايثون.

لقد وضعنا أيضًا في هذا الفصل كيفية تحويل العديد من أنواع البيانات الأصلية المهمة إلى

أنواع بيانات أخرى، وذلك باستخدام التوابيع المُضمَّنة. يوفر تحويل أنواع البيانات في بايثون لك

مرونةً إضافيةً في مشاريعك البرمجية. يمكنك التعرف على المزيد من التفاصيل عن [هذه الأنواع](#) وطرائق التحويل بينها في موسوعة حسوب.